

Evaluating Generated Commit Messages with Large Language Models

Qunhong Zeng
Beijing Institute of Technology
Beijing, China
qunhongzeng@bit.edu.cn

Yuxia Zhang*
Beijing Institute of Technology
Beijing, China
yuxiazh@bit.edu.cn

Zexiong Ma
Peking University
Beijing, China
mazexiong@stu.pku.edu.cn

Bo Jiang
ByteDance
Shenzhen, China
jiangbo.jacob@bytedance.com

Ningyuan Sun
ByteDance
Shenzhen, China
sunningyuan@bytedance.com

Klaas-Jan Stol
University College Cork
Cork, Ireland
k.stol@ucc.ie

Xingyu Mou
Beijing Institute of Technology
Beijing, China
xingyum@bit.edu.cn

Hui Liu
Beijing Institute of Technology
Beijing, China
liuhui08@bit.edu.cn

Abstract

Commit messages are essential in software development as they serve to document and explain code changes. Yet, their quality often falls short in practice, with studies showing significant proportions of empty or inadequate messages. While automated commit message generation has advanced significantly, particularly with Large Language Models (LLMs), the evaluation of generated messages remains challenging. Traditional reference-based automatic metrics like BLEU, ROUGE-L, and METEOR have notable limitations in assessing commit message quality, as they assume a one-to-one mapping between code changes and commit messages, leading researchers to rely on resource-intensive human evaluation. This study investigates the potential of LLMs as automated evaluators for commit message quality. Through systematic experimentation with various prompt strategies and state-of-the-art LLMs, we demonstrate that LLMs combining Chain-of-Thought reasoning with few-shot demonstrations achieve near human-level evaluation proficiency. Our LLM-based evaluator significantly outperforms traditional metrics while maintaining acceptable reproducibility, robustness, and fairness levels despite some inherent variability. This work conducts a comprehensive preliminary study on using LLMs for commit message evaluation, offering a scalable alternative to human assessment while maintaining high-quality evaluation.

CCS Concepts

• **Software and its engineering** → **Collaboration in software development; Programming teams.**

*Corresponding authors

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
ICSE '26, Rio de Janeiro, Brazil
© 2026 Copyright held by the owner/author(s).
ACM ISBN xxx
<https://doi.org/xxx>

Keywords

Commit Message Generation, Commit Message Quality, Metrics, Large Language Models (LLMs), Evaluation

ACM Reference Format:

Qunhong Zeng, Yuxia Zhang, Zexiong Ma, Bo Jiang, Ningyuan Sun, Klaas-Jan Stol, Xingyu Mou, and Hui Liu. 2026. Evaluating Generated Commit Messages with Large Language Models. In *48th International Conference on Software Engineering (ICSE '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 12 pages. <https://doi.org/xxx>

1. 背景：提交信息是软件开发的重要文档，高质量提交信息包含 What 和 Why。

1 Introduction

Commit messages serve as a communication medium between developers to document code changes, and are integral to software development with version control systems [6]. A high-quality commit message should contain two elements: the ‘What,’ a summary of what changed, and the ‘Why’: the rationale behind these changes. These two elements help developers to quickly grasp code changes without having to dive too much into implementation details and the commit history to understand the reason behind specific code changes [23, 41].

Despite the generally acknowledged importance of clear commit messages, developers may fail to write high-quality messages because of tight deadlines or a lack of motivation [33]. This mismatch between best practices and reality is underscored by prior findings that over 14% of commit messages in SourceForge-hosted projects were empty [12], while nearly 44% of commit messages in high-activity open source projects demanded significant quality improvements [41]. These gaps in commit message quality pose considerable risks to project sustainability, as incomplete or poorly written messages hinder effective collaboration and complicate code comprehension [41]. Consequently, researchers have invested substantial efforts in automating commit message generation (CMG) to reduce developers’ workload while improving the overall quality of commit messages.

2. 问题：实际开发中提交信息质量差，影响协作与理解。Recently, significant advances have been made in automatic CMG [10, 27, 31–33, 49], evolving from early rule-based approaches to leveraging Large Language Models (LLMs) [24, 50, 52]. Researchers

3. 现有方法缺陷：传统指标不可靠，人工评估成本高。

typically evaluate the quality of these approaches using reference-based automatic metrics, including BLEU [34], ROUGE-L [25], and METEOR [3]. These reference-based metrics evaluate generated messages by computing their textual similarity with developer-written reference messages. However, this approach has three significant limitations. First, evaluation scores become unreliable when reference messages are of lower quality than generated ones. Second, these metrics capture *textual* rather than *semantic* similarities, failing to recognize messages that convey the same meaning but using different expressions. Third, different developers may write semantically distinct yet equally valid commit messages for the same diff. For example, to summarize changes in a commit, while some developers might compare implementation alternatives, others might focus on describing the functionality [41]. In such cases, generated messages might effectively describe code changes but receive low scores due to limited textual or semantic overlap with the developer-written reference message. These limitations have been acknowledged, and researchers have incorporated human evaluation to provide more comprehensive assessments [10, 24, 38, 45]. For instance, Li et al. [24] evaluated commit messages across four dimensions: Comprehensiveness (how good commit messages describe a summary of what has been changed), Rationality (how good commit messages provide a logical explanation for the code changes), Conciseness, and Expressiveness. They found that their approach scored lower than the baseline on BLEU and ROUGE-L metrics, but performed better in human evaluation. Again, this reveals that the widely used reference-based metrics may be unreliable in the evaluation of generated commit message.

Although human evaluation remains the gold standard for assessing commit message quality, human evaluation is time-consuming, resource-intensive, and therefore lacks scalability. Reproducibility is also challenging due to evaluator availability and consistency issues. Recent advances in generative AI technology [7, 29] have demonstrated the effectiveness of LLMs in evaluating natural language processing (NLP) tasks that require diversity and creativity, such as open-ended story generation [7] and dialogue generation [29]. These promising results suggest the potential of using LLMs to evaluate generated commit messages, thereby overcoming the limitations associated with human evaluation. To the best of our knowledge, how LLMs perform in evaluating commit messages, and whether they are comparable to human evaluators, remain open questions. Thus, this paper presents the results of the first empirical investigation of LLM-based evaluation of commit messages, guided by three research questions.

First, we investigated whether LLM-based evaluators can perform comparably to human evaluators in assessing commit messages. Thus, we ask **RQ1: Are LLM-based evaluators comparable to human evaluators in commit message evaluation?** We leverage an LLM-based evaluator to assess commit messages and use Spearman [39] and Kendall [19] correlation coefficients to evaluate the correlation between the LLM-based evaluator and human developers. Apart from directly prompting LLMs to perform the commit message evaluation task, utilizing their zero-shot in-context learning capability through carefully constructed prompts that incorporate evaluation task descriptions, assessment criteria, and target examples, we also explore various prompting strategies

4. 研究问题：LLM 能否替代人工评估提交信息？本文首次系统研究。

to enhance LLMs' zero-shot learning performance. We systematically examine combinations of different prompting techniques, including Chain-of-Thought reasoning (providing structured evaluation steps) [20], few-shot demonstrations [42], and alternative scoring approaches (unified multi-dimensional assessment within a single prompt versus separate evaluation of individual dimensions). Our experiments use multiple state-of-the-art LLMs, including proprietary model GPT-4 [1] and open-source ones (Llama 3.3 [11], Qwen 2.5 [51], QwQ [40]). The experimental results reveal that an LLM-based evaluator with an appropriate prompting strategy correlates highly with human evaluators, indicating comparable performance in commit message evaluation.

Since LLM-based evaluators exhibit comparable evaluation performance to human developers' assessments, it is of interest to explore how LLM-based evaluators compare with traditional automatic evaluation metrics commonly used in the CMG literature. Thus, we ask **RQ2: What is the performance of the LLM-based evaluator compared to automatic evaluation metrics?** We implemented the optimal prompt strategy from RQ1 as our LLM-based evaluator. Our comparison encompasses both established metrics from CMG literature (BLEU [34], ROUGE-L [25], METEOR [3], CIDEr [44]) and widely-adopted NLP metrics previously unexplored in commit message generation, BERTScore [53] and SBERT (Sentence BERT) [35]. Analysis of Spearman [39] and Kendall [19] correlation coefficients reveals that the LLM-based evaluator outperforms traditional automatic metrics in commit message quality assessment by a large margin, demonstrating superior alignment with human evaluation. Through thematic analysis [9], we further investigate the underlying factors contributing to the limited correlation between commonly used reference-based metrics and human evaluation.

Despite demonstrating comparable performance to human evaluators and outperforming traditional widely-used metrics by a large margin, LLMs' non-deterministic behavior in commit message evaluation raises stability concerns. We sought to investigate three common concerns about reproducibility, robustness, and fairness. Thus, we ask **RQ3: Can the performance of the LLM-based evaluators keep stable?** Reproducibility examines evaluation stability for identical inputs by analyzing scoring consistency across multiple evaluations with controlled temperature settings. Robustness assesses stability across semantically equivalent commit messages with different expressions. Fairness investigates potential biases in LLM-based evaluators. While LLMs exhibit some scoring variability, they demonstrate acceptable reproducibility and robustness without significant discriminatory bias.

The main contributions of this work are as follows:

- (1) We present the first investigation of utilizing LLMs for commit message quality evaluation, exploring effective implementation strategies and demonstrating strong correlations with human developer assessments.
- (2) We demonstrate that the LLM-based evaluator significantly outperforms widely used reference-based automatic metrics.
- (3) We conduct a comprehensive stability analysis of LLM-based commit message evaluation, examining its reproducibility, robustness, and fairness.

5. 贡献：首次用 LLM 做提交信息评估，效果超传统指标，验证稳定性。

We provide a replication package to facilitate reproducibility and future work: <https://anonymous.4open.science/r/4k8asu>.

2 Background

In this section, we first introduce the task and related works of commit message generation in Sec. 2.1. Next, we discuss two widely used evaluation methodologies in commit message generation research: automatic metrics evaluation (Sec. 2.2) and human evaluation (Sec. 2.3). Finally, Sec. 2.4 presents background on LLMs and techniques for enhancing their zero-shot learning capabilities.

2.1 Commit Message Generation

Commit message generation (CMG) is an automated software engineering task that aims to generate descriptive and informative commit messages for code changes in version control systems. While most CMG approaches generate messages based primarily on code diffs (changes between two versions of code), some incorporate additional inputs such as related issues and pull requests. Existing CMG approaches can be categorized into four types: rule-based, retrieval-based, learning-based, and hybrid approaches [55].

Rule-based approaches [5, 8, 37] rely on predefined patterns to analyze code modifications. When changes match these patterns, corresponding template-based messages are generated. However, these methods have limited capability to explain the rationale behind changes and are inflexible if modifications can not be aligned with the predefined patterns. Retrieval-based approaches [14, 30] identify similar code changes in existing datasets, then adopt their corresponding developer-written messages. Their effectiveness depends heavily on the similarity search algorithm. Moreover, even seemingly similar code changes may not justify identical commit messages when the underlying context differs. Learning-based approaches [10, 17, 27, 31–33, 49] frame CMG as a machine translation task, employing various methods to represent code changes. For instance, CoreGen [33] utilizes a transformer architecture to represent code changes as token sequences. FIRA [10] performs fine-grained graph representation by parsing the Abstract Syntax Tree for each change hunk, utilizing both edit operations and sub-tokens. Finally, hybrid approaches [24, 28, 38, 47] combine retrieval-based and learning-based methods. For example, RACE [38] retrieves similar commits based on code change similarity and incorporates both retrieved code changes and messages into the generation process. OMG [24] enhances generation by retrieving related Pull Request titles and issue reports as additional context, combining these with code changes and utilizing GPT4 for message generation, achieving state-of-the-art performance.

The quality of generated messages in prior studies is evaluated through two primary methods: automatic metrics evaluation and human evaluation. Table 1 summarizes these evaluation methods.

2.2 Automatic Metrics Evaluation for CMG

Among prior CMG studies, four automatic metrics have been used to evaluate the quality of generated commit messages: BLEU [34], ROUGE [25], METEOR [3], and CIDEr [44]. This section briefly introduces these metrics since they are widely used and the formulas have been well documented. We provide a detailed introduction in our online appendix [2].

BLEU is a widely adopted precision-based metric. It calculates the precision for different n-gram lengths (typically 1-gram to 4-gram) and then takes a weighted geometric average of these precision scores.

ROUGE comprises a series of variant metrics [25]. The two most frequently used variants are ROUGE-N and ROUGE-L. ROUGE-N measures the n-gram recall between the generated and reference messages, whereas ROUGE-L computes the length of the longest common subsequence. ROUGE-L is widely used in commit message generation literature.

METEOR is an evaluation metric based on the harmonic mean of unigram precision and recall, with additional linguistic features like stemming, synonym matching, and word order [3]. In calculating precision and recall, METEOR considers not only exact unigram matches but also stem matching and synonymy matching.

CIDEr is an evaluation metric designed initially for image caption evaluation [44], which has been introduced to evaluate commit message quality by Shi et al. [38]. It measures the similarity between generated and reference commit messages using TF-IDF weighted n-gram matching.

2.3 Human Evaluation for CMG

As we argued in Sec. 1, automatic reference-based evaluation metrics may not adequately assess the quality of generated commit messages, because developers may write semantically distinct yet equally valid commit messages for the same diff. Consequently, several studies [5, 8, 10, 17, 24, 28, 30, 31, 37, 38, 47] have incorporated human evaluation for further quality assessment. Early approaches such as NNGen [30], ContextNMT [31], and FIRA [10] used a 5-point scale for developers to rate the semantic similarity between generated messages and their references, while CmtGEN

Table 1: Overview of evaluation metrics used in commit message generation approaches

Approach	Automatic				Human
	BLEU	ROUGE	METEOR	CIDEr	
DeltaDoc [5]					✓
ChangeScribe [8]					✓
AutoSumCC [37]					✓
NNGen [30]	✓				✓
LogGen [14]	✓				
CmtGen [17]	✓				✓
NMT [32]	✓				
ContextNMT [31]	✓		✓		✓
CODISUM [49]	✓		✓		
PtrGNCMsg [27]	✓	✓			
CoreGen [33]	✓	✓	✓		
FIRA [10]	✓	✓	✓		✓
RACE [38]	✓	✓	✓	✓	✓
CoRec[47]	✓	✓	✓		✓
ATOM [28]	✓	✓	✓		✓
OMG [24]	✓	✓	✓		✓

[17] adopted an 8-point scale, where higher scores indicate greater similarity to the reference messages.

Although human developers can understand semantic similarities between different messages, reference-based evaluation approaches have inherent limitations. Developers may compose different yet equally valid messages for the same code changes [41], making similarity to the reference insufficient for evaluation. Recent works have shifted toward multi-dimensional assessment using 5-point Likert scales without references. For instance, CoRec [47] evaluates messages along three dimensions: *relevance*, *usefulness*, and *adequacy*. Similarly, RACE [38] and HORDA [46] assess *informativeness*, *conciseness*, and *expressiveness*, while OMG analyzed existing human evaluation metrics and consolidated them into four dimensions: *comprehensiveness* (the effectiveness of describing code changes, i.e., *What*), *rationality* (the quality of explaining change rationale, i.e., *Why*), *conciseness*, and *expressiveness* [24].

Among these dimensions, *conciseness* and *expressiveness* focus on linguistic quality, where current generation methods demonstrate strong performance. For instance, Wang et al. [46] reports that RACE [38] achieves average scores of 4.86 and 4.82 out of 5 in these aspects, respectively. In light of these high scores, and considering that our study requires a larger annotation volume than [46], we focus on content-oriented metrics and exclude language quality. According to Tian et al. [41], commit message quality fundamentally comprises *What* and *Why*. The evaluation metrics of OMG [24] align with this definition of a good commit message, where *comprehensiveness* and *rationality* can be seen as specific sub-dimensions of *informativeness*. Therefore, this study's human evaluation specifically targets these two fundamental aspects of commit message content quality: *What* and *Why*.

2.4 Large Language Models

Large Language Models (LLMs) harness billions of parameters and are pre-trained on extensive natural language corpora. While proprietary models like GPT-4 [1] initially dominated the field, recent open source alternatives such as the DeepSeek family [26], Qwen series [51], and LLaMA variants [11] have demonstrated comparable capabilities, gradually narrowing the performance gap across various benchmarks such as coding. These models have been widely adopted in software engineering tasks, such as code generation [18] and also commit message generation [24]. Built on multi-layer transformer architectures [43], these models have shown that scaling, either through increased model parameters or expanded training data, unlocks remarkable abilities in tackling complex tasks [56]. LLMs exhibit exceptional performance on unseen tasks, allowing them to perform tasks given only instructions without task-specific training effectively; this kind of ability is called zero-shot in-context learning [7].

Various techniques exist to enhance LLMs' zero-shot in-context learning performance. One common approach involves additional training or fine-tuning on target tasks, which requires appropriate available data and computational resources. Instead of further training, LLMs' in-context learning capabilities can be improved through prompting strategies, such as few-shot learning [4] or chain of thought (CoT) reasoning [48]. Few-shot learning enhances model performance on specific tasks by providing a few input-output

exemplars in the prompt. CoT simulates human problem-solving processes by incorporating step-by-step logical deductions. This approach includes not only the problem description in the prompt but also guides the model to generate intermediate reasoning steps, forming a logical chain from the problem to its solution to enhance target task performance.

In this study, we employ various state-of-the-art LLMs including GPT-4 [1], Llama-3.3-70B-Instruct [11], Qwen2.5-72B-Instruct [51], and QwQ-32B [40]. Among these, QwQ-32B is a lightweight reasoning model (comprising only 32 billion parameters) similar to DeepSeek-R1 [13], which is trained to provide a reasoning process before delivering task answers to improve problem-solving performance. We test these models with different prompting strategy combinations to evaluate their capabilities in assessing commit message quality.

3 Dataset

We constructed a benchmark dataset comprising code changes, developer-written commit messages, and corresponding generated commit messages derived from the code changes. For each commit, both the developer-written and generated messages were evaluated by human developers. This section describes our data selection and human evaluation protocol.

3.1 Data Preparation

3.1.1 Sample. Given the resource-intensive nature of human evaluation and the complexity of recruiting annotators proficient in multiple programming languages, we sought to balance data diversity with feasibility by focusing on two widely used languages: Python and Java. Our data source is the CommitBench dataset [36], which considers redistribution licenses and enforces various quality filters, including removing bot-generated messages and privacy-sensitive content. We randomly sampled 200 Python commits and 200 Java commits from CommitBench_long¹, which uses an extended token limit of 2,048 instead of the default 512², as Zhang et al. [55] found a median count of 632 tokens in diffs. Since our evaluation mainly focuses on commit message content, we excluded auxiliary information such as issue links and other URLs. While CommitBench attempts to sanitize issue references by replacing GitHub issue tags with the label <I>, this approach is limited to GitHub's issue format and does not address references from other issue tracking platforms or custom issue tags. Additionally, some personal names or other privacy-sensitive data remained in the dataset. To this end, we manually reviewed and processed each sampled developer-written message by:

- (1) Removing issue tags (both CommitBench's <I> replacements and unprocessed custom issue tags) and URLs.
- (2) Eliminating personal names and email addresses.
- (3) Excluding revert commits (replacing them with newly sampled non-revert commits).

3.1.2 Generate Messages for the Sampled Commits. To address RQ2, we aim to compare LLM-based evaluators with commonly used automatic reference-based metrics, such as METEOR and ROUGE.

¹https://huggingface.co/datasets/Maxscha/commitbench_long

²<https://huggingface.co/datasets/Maxscha/commitbench>

Thus, we need to prepare generation messages for the 400 selected commits. To ensure a comprehensive evaluation of the metrics' ability to discriminate messages with diverse qualities, we intentionally introduced quality variation in the generated messages. Previous work that used LLMs for commit message generation [24, 50, 52] shows that models of different sizes can exhibit varying capabilities. Therefore, we employed three LLMs of different scales: GPT-4 [1], Llama-3.3-70B-Instruct [11], and Qwen-2.5-14B-Instruct [51], with each model generating one-third of the commit messages. However, our preliminary annotation results showed that most LLM-generated messages received high scores (≥ 3) on the *What* dimension. To ensure a wide quality range in our benchmark, we deliberately instructed the models to generate poor-quality commit messages for 20% of randomly selected samples. Details of the implementation and prompts are available in the appendix [2].

3.2 Human Evaluation

The human evaluation focuses on two fundamental dimensions of commit message content quality: *What* and *Why*. The definitions of these two dimensions are [24, 41]:

- (1) **What:** How well the commit message accurately captures the changes made in the code.
- (2) **Why:** How well the commit message explains the rationale behind making the change.

We recruited six volunteers, each with over five years of software development experience and Python and Java programming proficiency. From our dataset of 400 commits, each code change was associated with two commit messages: one written by the original developer and one generated by an LLM. Similar to previous work [10, 24, 31, 38], we used a 5-point Likert scale (0 for poor, 1 for marginal, 2 for acceptable, 3 for good, and 4 for excellent). To ensure the reliability and consistency of the evaluation, we conducted a three-stage annotation process. First, in the pilot annotation stage, each volunteer independently assessed 50 identical commit samples. The volunteers were granted access to the commit's git repository to facilitate a good understanding of the commits under evaluation, scoring both the *What* and *Why* dimensions for each commit message. This stage aimed to familiarize the volunteers with the evaluation task and establish an initial understanding of scoring criteria. Second, after the pilot annotation stage, we conducted a discussion and calibration session where the volunteers explained the reasons behind their different scores for the same commit message. This stage allowed the volunteers to learn from each other, understand different perspectives, and reach a consensus on the scoring criteria. Finally, in the formal evaluation stage, the volunteers independently assessed the remaining 750 commit messages. To minimize potential bias, we randomly shuffled the evaluation samples. Each volunteer received randomly shuffled commits (code changes, messages, and git repository information) for evaluation, without knowing whether the commit messages were human-written or LLM-generated.

To assess the inter-rater reliability of the six volunteers for both the *What* and *Why* dimensions, we calculated Spearman correlation coefficients [39] between each pair of volunteers' scores. The results are shown in Figure 1. All correlation coefficients exceeded 0.6, indicating a high agreement [21] among volunteers. The final

What and *Why* scores for each commit message were computed by averaging all volunteers' rating scores.

v1	1.00	0.64	0.63	0.65	0.67	0.67
v2	0.68	1.00	0.69	0.65	0.73	0.72
v3	0.65	0.78	1.00	0.67	0.71	0.63
v4	0.65	0.67	0.66	1.00	0.70	0.62
v5	0.74	0.78	0.78	0.67	1.00	0.66
v6	0.61	0.68	0.62	0.66	0.69	1.00
	v1	v2	v3	v4	v5	v6

Figure 1: Spearman Correlation Coefficient of Volunteers, the upper triangle (blue) represents *What*, the lower triangle (green) represents *Why*

3.3 Implementation Details

We used various LLMs in our experiments. For the proprietary model GPT-4 (gpt-4-0613), we utilized the Azure API to send requests. For open source models, we deployed the backend using vLLM [22] on a cluster of 8x NVIDIA A100 40GB GPUs. Based on empirical performance evaluation of different temperature settings (detailed in Sec. 6.1), we set the temperature parameter at 0.25 for proprietary models and 0 for open-source models. The top_p parameter remained at its default value of 0.5 across all experiments. To increase reliability, final LLM scores for all experiments were derived by averaging three repeated evaluations of identical inputs. For automatic metrics implementation, we used the Evaluate library [16] from the Hugging Face community to ensure implementation reliability. We implemented BertScore using the BertScore library [54] with its default roberta-large model, and Sentence-BERT Score using the sentence-transformers library with the all-MiniLM-L6-v2 model.³ The CIDEr metric was implemented using the code provided by Shi et al. [38].

4 RQ1: Are LLM-based evaluators comparable to human evaluators in commit message evaluation?

4.1 Method

To evaluate commit message quality using LLMs, we first employed zero-shot in-context learning, assessing the *What* and *Why* dimensions using a 5-point Likert scale. The prompt provided to the LLM is: “You are an experienced software engineer. Your task is to evaluate the commit message based on the corresponding git commit diff, focusing on two dimensions: <dimensional definitions> + <scoring criteria> + <commit to be evaluated>.” Within this prompt, we included

³<https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>.

explicit definitions of each dimension, detailed scoring criteria, and both the commit message under evaluation and its corresponding code changes.

Building upon the introduction of LLMs in Sec. 2.4, we then investigated three prominent strategies to enhance zero-shot in-context learning performance: Chain of Thought [48], Few-Shot Learning [4], and their combination. In the Chain of Thought approach, we instructed the LLM to evaluate commit messages step-by-step. In the Few-Shot Learning approach, we provided the LLM with exemplary high- and low-quality examples to illustrate the evaluation criteria clearly. Due to space constraints, detailed descriptions of all prompts are provided in the online appendix [2]. Additionally, we explored two scoring methodologies: unified evaluation, in which the LLM assesses both dimensions within a single prompt, and separate evaluation, in which each dimension is assessed individually. The aim was to identify the strategy most closely aligned with human evaluations.

We evaluated four state-of-the-art LLMs: GPT-4 [1], Llama-3.3-70B-Instruct [11], Qwen2.5-72B-Instruct [51], and QwQ-32B [40]. Each LLM was tested using various combinations of prompting strategies to determine the optimal model and prompt configuration. We use 400 human-written commit messages from the dataset constructed in Section 3. For each combination of LLM and prompting strategy, suitable assessment prompts were constructed. To quantify the alignment between LLM-generated and human evaluator ratings, we calculated Spearman [39] and Kendall [19] correlation coefficients, which are appropriate measures for paired data, following established methodologies from recent studies [7, 15, 29].

4.2 Results

Table 2 presents the results, where “separate” indicates separate evaluations of the *What* and *Why* dimensions, while “unified” represents a single evaluation process assessing both dimensions simultaneously. Most models demonstrated significant improvement in performance under unified evaluation compared to separate evaluation in the *Why* dimension, with minor improvements or slight fluctuations in the *What* dimension.

Our analysis of different prompting strategies revealed several findings. First, the Chain of Thought (CoT) strategy effectively improved correlation with human judgment in the *Why* dimension across both evaluation approaches. For instance, GPT-4’s Spearman correlation increased from 0.58 to 0.70 in separate evaluation and from 0.64 to 0.78 in unified evaluation. However, the CoT improvement in the *What* dimension was limited, with some models showing slight improvements while others exhibited minor decreases. Notably, QwQ-32B maintained nearly unchanged performance between zero-shot and CoT conditions, as this reasoning model inherently outputs a reasoning process (CoT) regardless of whether CoT prompts are explicitly provided. The Few-shot strategy demonstrated consistent effectiveness across both dimensions, with all models benefiting from or maintaining performance levels in both evaluation settings.

When combining CoT and Few-shot strategies, most models maintained the improvements in the *Why* dimension achieved through CoT, while performance in the *What* dimension varied slightly among models, generally remaining at comparable levels.

Based on these results, we selected GPT-4 with the CoT+Few-shot strategy using unified evaluation as our optimal configuration, achieving Spearman correlations of 0.65 and 0.78 for the *What* and *Why* dimensions, respectively. Notably, these correlation coefficients closely match the inter-rater correlations among human developer volunteers (see Figure 1), suggesting that GPT-4 with appropriate enhancements can achieve human-level proficiency in commit message evaluation.

Summary for RQ1: LLMs demonstrate comparable evaluation performance to human developers when evaluating commit messages across the dimensions of *What* and *Why*. Among the four LLMs of varying sizes and types evaluated in our study, even the lowest-performing model achieved a Spearman correlation of approximately .53, indicating a moderate to strong correlation. The correlation between LLM and human developer ratings can be further enhanced by selecting appropriate base models and implementing suitable prompting strategies. The best-performing model achieved Spearman correlations of .65 for the *What* dimension and .78 for the *Why* dimension, demonstrating human-level proficiency in commit message evaluation.

5 RQ2: What is the performance of the LLM-based evaluator compared to automatic evaluation metrics?

5.1 Method

This research question asks how well LLM-based evaluators perform when assessing generated commit messages, and how they compare to traditional, commonly used automatic evaluation metrics. To conduct a comprehensive comparison of evaluation metrics, we incorporated all metrics previously used in commit message generation studies, including BLEU, ROUGE, METEOR, and CIDEr (see Sec. 2.2 for their underlying principles and computational methods). Additionally, we introduced two semantic-based metrics, i.e., BERTScore [53] and SBERT (Sentence BERT) [35], which are not previously used in commit message evaluation but widely adopted in NLP tasks.

BERTScore is a text evaluation metric that measures semantic similarity by computing token-level cosine similarities using BERT embeddings [53]. It creates a similarity matrix between candidate and reference texts, then uses greedy matching to calculate precision (matching candidate tokens to reference) and recall (matching reference tokens to the candidate). The final score is the F1 measure of these matches, enabling a more robust evaluation than traditional string-matching metrics. Sentence-BERT (SBERT) [35] is a variant of BERT that creates sentence embeddings by converting text into fixed-length vectors. It processes each sentence independently through BERT and uses pooling to combine token embeddings into a single sentence vector. The similarity between two texts can be easily calculated using cosine similarity between their vectors.

Our evaluation involved 400 generated commit messages (see Sec. 3.2), assessed using both LLM-based and automatic metrics. We used GPT-4 with CoT+Few-shot prompting strategy under unified evaluation (see Sec. 4.2). Following established practices in commit message generation research, we used developer-written messages

Table 2: Correlation between LLM-generated and Human Evaluation Scores

Model	Strategy	What (Separate)		Why (Separate)		What (Unified)		Why (Unified)	
		Spearman	Kendall	Spearman	Kendall	Spearman	Kendall	Spearman	Kendall
GPT4	Zero-shot	0.61	0.49	0.58	0.47	0.66	0.54	0.64	0.52
	CoT	0.62	0.48	0.70	0.56	0.65	0.52	0.78	0.66
	Few-shot	0.63	0.51	0.65	0.53	0.63	0.52	0.68	0.55
	Zero-shot+CoT	0.63	0.50	0.72	0.57	0.65	0.52	0.78	0.64
QwQ-32B	Zero-shot	0.53	0.41	0.57	0.45	0.59	0.46	0.70	0.57
	CoT	0.54	0.42	0.62	0.49	0.60	0.46	0.72	0.60
	Few-shot	0.57	0.46	0.63	0.50	0.63	0.51	0.75	0.62
	Zero-shot+CoT	0.59	0.47	0.64	0.50	0.60	0.48	0.74	0.61
Llama-3.3-70B-Instruct	Zero-shot	0.61	0.50	0.55	0.45	0.61	0.50	0.66	0.55
	CoT	0.55	0.44	0.64	0.51	0.56	0.44	0.74	0.60
	Few-shot	0.62	0.51	0.56	0.45	0.62	0.52	0.68	0.56
	Zero-shot+CoT	0.61	0.49	0.63	0.49	0.57	0.46	0.71	0.57
Qwen2.5-72B-Instruct	Zero-shot	0.66	0.55	0.55	0.45	0.63	0.53	0.56	0.46
	CoT	0.58	0.48	0.60	0.48	0.61	0.50	0.60	0.49
	Few-shot	0.66	0.55	0.56	0.46	0.61	0.51	0.62	0.51
	Zero-shot+CoT	0.60	0.50	0.59	0.48	0.59	0.49	0.67	0.54

as references for automatic metrics. To assess metric alignment with human judgments, we calculated Spearman and Kendall correlation coefficients, consistent with our approach for addressing RQ1.

5.2 Results

Table 3: Correlation between Different Evaluation Metrics and Human Judgments

Metrics	What Dimension		Why Dimension	
	Spearman	Kendall	Spearman	Kendall
<i>Textual-Similarity Metrics</i>				
BLEU	0.08	0.07	0.10	0.09
ROUGE-L	0.28	0.20	0.07	0.05
METEOR	0.42	0.31	0.20	0.14
CIDEr	0.24	0.17	0.09	0.06
<i>Semantic-Similarity Metrics</i>				
BERTScore	0.21	0.15	0.12	0.08
SBERT	0.34	0.25	0.18	0.13
<i>LLM-based Metrics</i>				
GPT-4	0.65	0.56	0.78	0.63

Table 3 presents the evaluation results, with the best-performing metrics highlighted in bold. The results show that widely used textual and semantic similarity metrics perform poorly on the *Why* dimension, with the highest Spearman correlation coefficient being only .20 and the highest Kendall correlation coefficient being .14, indicating a weak correlation with human developers’ judgments [21, 39]. In the *What* dimension, BLEU exhibits the weakest correlation with human assessments, attributable to its design as a corpus-level metric, which compromises reliability in sentence-level evaluations of generated commit messages [34]. Nonetheless,

we include BLEU results for reference. Of the commonly used metrics, METEOR achieves superior performance with Spearman and Kendall correlations of .42 and .31, followed by SBERT with correlations of .34 and .25. Despite improved performance in the *What* dimension, even METEOR demonstrates only moderate correlation with human judgments [21, 39]. In contrast, LLM-based metrics, designed for distinct dimensional evaluation, achieve robust correlations in both dimensions when assessing generated commit messages, with Spearman correlations reaching .65 for *What* and .78 for *Why*, substantially surpassing all automatic metrics.

To further investigate the performance gap between LLM-based and reference-based automatic metrics in commit message evaluation, we conducted a thematic analysis [9], focusing on METEOR, the best-performing automatic metric. Specifically, our analysis examined discrepancies between human and METEOR evaluations, identifying 101 examples that received high human ratings (top 30%) for the *What* dimension but low METEOR scores (bottom 30%), and conversely, 1 example with a high METEOR score (top 30%) but low human rating (bottom 30%). Notably, we did not find any similar discrepancies with LLM metrics under the same filtering threshold, revealing the performance advantage of LLM metrics. We only considered the *What* dimension because the correlation between all reference-based automatic metrics and human developers is low for the *Why* dimension. Through a thematic analysis of these discrepancies and examining METEOR’s computational principles (Sec. 2.2), we investigated why METEOR assigned low scores to commit messages that human evaluators rated highly. We identified four main categories:⁴ Low-quality References, High-Expectation References, Lexical and Stylistic Variability, and Semantic Divergence. We discuss these next.

Category 1: Low-quality References (#32) refers to vague or incomplete developer-written messages (e.g., “fix a bug”), which

⁴Note that the sum of the instances in the four identified categories does not equal 102 because some instances belong to multiple categories.

exist in our dataset due to random sampling. In these cases, generated messages often surpass reference quality but share minimal vocabulary, resulting in low recall and, consequently, low METEOR scores despite high human ratings. This category is a common drawback for reference-based metrics. To address this issue, we selected a subset of the dataset where each reference’s *What* dimension human score was greater than or equal to 2.5, indicating a high-quality score. This filtering process left 236 cases; Table 4 presents the correlation comparison. The results show that the correlation between reference-based metrics and human judgments improved for the high-quality sub-dataset. METEOR remains the best-performing reference-based metric; however, its correlation still lags behind LLM-based metrics.

Table 4: Correlation between Different Evaluation Metrics and Human Judgments on High-quality Subset

Metrics	What Dimension		Why Dimension	
	Spearman	Kendall	Spearman	Kendall
<i>Textual-Similarity Metrics</i>				
BLEU	0.09	0.08	0.12	0.10
ROUGE-L	0.35	0.25	0.09	0.06
METEOR	0.53	0.38	0.27	0.19
CIDEr	0.31	0.22	0.12	0.08
<i>Semantic-Similarity Metrics</i>				
BERTScore	0.31	0.22	0.14	0.10
SBERT	0.40	0.29	0.16	0.11
<i>LLM-based Metrics</i>				
GPT-4	0.67	0.58	0.78	0.64

Category 2: High-Expectation References (#11) are messages that receive high scores from human developers and contain not only *What* but also *Why* information, as well as details that developers consider necessary to help readers understand the code changes. In such cases, the reference token count increases, resulting in a large recall denominator, requiring the generated message to include the same details. Although such references are the ultimate goal of automatic commit message generation, a generated message that does not include the same *Why* information and details as the reference does not necessarily indicate a low-quality message in terms of the *What* dimension.

Category 3: Lexical and Stylistic Variability (#22) refers to cases where the generated and developer messages express the same semantics. However, there can be various ways to express the same meaning. Because of the expression diversity, the two messages may have little word overlap, leading to low recall and, consequently, a low METEOR score. For example, one reference message of a code change in our analyzed cases is “[Tiny]Remove duplicated assignment,” while the generated message is “Remove redundant GcsClient instantiation.” Although the two messages express similar semantics, they only share one matching word: “Remove.”

Category 4: Semantic Divergence (#45) is the most significant and fundamental reason why reference-based metrics fail

to evaluate commit messages effectively. Essentially, the mapping from code changes to commit messages is not one-to-one. As identified by Tian et al. [41], the *What* and *Why* dimensions can be expressed through various perspectives. For example, in one of our analyzed cases, the reference commit message, “Added hash_create option so hashes can create new threads,” describes the modification from a functional perspective. The corresponding generated commit message, “Add hash_create parameter to add_comment and update process_hashes call,” summarizes the change in terms of the modified code objects. These different but valid perspectives create semantic divergence, resulting in minimal lexical and semantic similarity despite both messages can be high-quality.

Summary for RQ2: LLM-based metrics significantly outperform traditional reference-based automatic metrics in assessing generated commit messages. The thematic analysis of examples that received high human but low METEOR scores reveals four categories explaining why reference-based metrics perform poorly in evaluating commit messages. The primary reason is the one-to-many nature of mapping code changes to messages, and using a single reference to calculate similarity is hindered by this diversity.

6 RQ3: Can the performance of the LLM-based evaluators keep stable?

Although LLM-based evaluators demonstrate superior correlation with human developer judgments, their non-deterministic nature may affect their performance stability. This RQ examines three critical aspects of LLM stability: reproducibility, robustness, and fairness.

6.1 Reproducibility

Reproducibility assesses the consistency of LLM-based evaluator scores across multiple evaluations of identical inputs (code changes and commit messages).

6.1.1 Method. The non-determinism of LLM-generated output stems from multiple sources, including system-level variations in hardware architectures and computing environments (e.g., non-deterministic CUDA operators). However, the primary source is the LLM’s sampling mechanism, which typically employs probabilistic token selection based on predicted logits, controlled by the temperature hyperparameter [4]. Temperature values (0-1) regulate token selection probability: lower values increase determinism by amplifying high-logit token probability, while higher values enhance randomness. To evaluate the temperature’s impact on evaluator reproducibility, we assessed 400 developer-written messages with five temperature settings (0, 0.25, 0.5, 0.75, and 1), conducting three evaluations per message at each setting. We tested four models from RQ1, all using the CoT+few-shot prompting strategy under unified evaluation. For each message at each temperature setting, we calculated the standard deviation (std) and mean absolute difference (MAD) across the three evaluations. We then averaged these metrics across all 400 messages to assess the overall evaluation consistency at each temperature value.

6.1.2 Results. The results in Table 5 demonstrate distinct stability patterns across LLM evaluators. Open-source models (QwQ, Qwen2.5, and Llama3.3) deployed via VLLM [22] exhibited progressively decreasing evaluation stability with increasing temperature values, while the proprietary model GPT-4 accessed through its API demonstrated comparable levels of variation across all temperature settings, with optimal performance at temperature 0.25. Taking GPT-4 at temperature 0.25 as an example, we analyzed 1,200 pairs of evaluations (derived from 400 triplets). In the *What* dimension (mean std=0.24), 906 (75.5%) pairs showed identical scores, with 258 pairs differing by one point and 36 pairs showing larger differences. The *Why* dimension (mean std=0.29) exhibited similar patterns, with 890 (74.2%) identical pairs, 215 one-point differences, and 95 pairs showing larger differences. Considering that a one-point scoring difference is generally acceptable (given that human developers may also face similar granularity in distinguishing between scores like 2 and 3), GPT-4 demonstrated stability in 97% of cases for the *What* dimension and 92% for the *Why* dimension. These results indicate that, in most cases, LLM-based evaluators can give stable scores for identical inputs. In scenarios that require high precision, we recommend using averaged scores from multiple repeated evaluations when employing LLMs for commit message assessment to increase result stability.

Table 5: Reproducibility analysis of different LLMs across temperature settings

Model	Dim	Metric	Temperature				
			0.00	0.25	0.50	0.75	1.00
GPT-4	What	Std.	0.29	0.24	0.26	0.26	0.28
		MAD	0.34	0.28	0.31	0.31	0.33
	Why	Std.	0.38	0.29	0.36	0.34	0.38
		MAD	0.46	0.35	0.43	0.40	0.45
QwQ	What	Std.	0.23	0.24	0.25	0.23	0.26
		MAD	0.27	0.28	0.29	0.27	0.31
	Why	Std.	0.27	0.29	0.29	0.28	0.31
		MAD	0.32	0.34	0.35	0.33	0.37
Llama-3.3	What	Std.	0.09	0.11	0.15	0.16	0.18
		MAD	0.10	0.13	0.18	0.19	0.21
	Why	Std.	0.24	0.27	0.29	0.32	0.31
		MAD	0.28	0.32	0.34	0.38	0.37
Qwen2.5	What	Std.	0.01	0.06	0.09	0.10	0.14
		MAD	0.01	0.07	0.11	0.11	0.17
	Why	Std.	0.02	0.09	0.15	0.16	0.20
		MAD	0.03	0.10	0.17	0.19	0.24

6.2 Robustness

Robustness examines score stability across semantically equivalent messages with varying expressions, such as “*Added support for Pine H64 board detection*” and “*Incorporate PineH64 into board detection*,” which should receive comparable scores despite different lexical expressions.

6.2.1 Method. This investigation examined LLM-based evaluators’ ability to maintain scoring consistency across semantically equivalent commit messages. For each of the 400 developer-written messages, we constructed a triplet consisting of the original message and two paraphrased variants, with the variants generated by GPT-4 (generation prompts and implementation details are provided in the appendix [2]). The first two authors manually verified semantic equivalence between the original messages and their variants, reaching a consensus on 342 triplets suitable for analysis. We employed LLM-based metrics three times for each message in these triplets and calculated the average as the final score. To assess consistency, we computed the standard deviation (std) and mean absolute difference (MAD) of final scores within each triplet, then averaged these metrics across all triplets. All LLM-based metric configurations followed the settings described in Sec. 6.1.

6.2.2 Results. Table 6 presents the average std and MAD for each model’s evaluation of the 342 semantically equivalent commit messages. The results indicate moderate variations in LLM evaluations across different expressions of the same semantic commit message. Among the four evaluated models, GPT-4 shows the least variation, with an average std of 0.29 for the *What* dimension and 0.25 for the *Why* dimension (the maximum score is 4). Across all pairwise combinations within the 342 triplets, GPT-4’s evaluation results show that 71.5% of score variations were less than or equal to 0.5 and 93.8% were less than or equal to 1 in the *What* dimension; in the *Why* dimension, 75.3% of score variations were less than or equal to 0.5 and 97% were less than or equal to 1. Considering that a one-point scoring difference is generally acceptable, GPT-4 demonstrates stability in 93.8% of cases for *What* and 97% for *Why* dimensions, suggesting acceptable robustness despite some variations.

Table 6: Robustness analysis of different LLMs

Model	What Dimension		Why Dimension	
	Std.	MAD	Std.	MAD
GPT-4	0.29	0.36	0.25	0.31
QwQ	0.27	0.33	0.33	0.41
Llama-3.3	0.28	0.33	0.44	0.54
Qwen2.5	0.24	0.28	0.40	0.48

6.3 Fairness

Fairness investigates potential bias in LLM-based evaluators, particularly whether they exhibit bias toward LLM-generated messages compared to human-written ones.

6.3.1 Method. We used 342 triplets from Sec. 6.2, each containing one developer-authored commit message and two semantically equivalent GPT-4-generated variants that deserved equivalent ratings on the *What* and *Why* dimensions. We analyzed the results from Sec. 6.2.2, calculating the average scores assigned by LLMs to both developer-authored and LLM-generated messages to investigate potential scoring bias when evaluating semantically equivalent messages with varying expressions. Additionally, since our RQ1 utilized GPT-4 to evaluate human-authored commit messages

Table 7: Evaluation scores comparison between human-authored and LLM-generated commit messages

Model	Human-authored		LLM-generated	
	What	Why	What	Why
GPT-4	2.74	1.42	2.81 ($\uparrow$ 0.08)	1.49 ($\uparrow$ 0.08)
QwQ	3.10	1.16	3.09 ($\downarrow$ 0.01)	1.18 ($\uparrow$ 0.02)
Llama-3.3	3.18	1.87	3.22 ($\uparrow$ 0.04)	1.98 ($\uparrow$ 0.11)
Qwen2.5	3.30	2.35	3.35 ($\uparrow$ 0.05)	2.45 ($\uparrow$ 0.09)

and RQ2 also used GPT-4 to evaluate LLM-generated messages. In our dataset, both human-authored and LLM-generated messages are assessed by human developers, we identified 25 pairs (human-authored, LLM-generated) that received identical human developer scores on the *What* dimension and 40 pairs with identical scores on the *Why* dimension. We also report GPT-4’s average scores for these pairs to investigate potential bias in its evaluation of human-authored versus LLM-generated messages.

6.3.2 Results. Table 7 presents mean evaluation scores assigned by different LLMs to semantically equivalent messages. The results indicate consistent evaluations across human-written and LLM-generated messages, with marginal differences favoring LLM-generated messages (e.g., GPT-4 shows increases of 0.08 in both *What* and *Why* dimensions). Furthermore, in our dataset, among the 25 pairs receiving identical human ratings on the *What* dimension, GPT-4 assigned an average score of 3.4 to human-authored messages and 3.72 to LLM-generated messages. For the 40 pairs with identical human ratings on the *Why* dimension, GPT-4 assigned an equal average score of 0.85 to both message types. Overall, we did not observe substantial bias in LLM metrics, though GPT-4 scored LLM-generated messages 0.32 points higher on average, potentially indicating bias. However, due to the limited sample size, more comprehensive research on LLM fairness remains for future work.

Summary for RQ3: Due to the inherent non-deterministic behavior of LLMs, the LLM-based commit message evaluators exhibit some variations in reproducibility (identical inputs) and robustness (semantically equivalent but differently expressed inputs). However, the overall reproducibility and robustness remain within acceptable ranges. Our analysis revealed no substantial bias between human-written and LLM-generated commit messages, although more comprehensive research on LLM fairness is required.

7 Discussion and Conclusion

7.1 Advantages and limitations of LLM-metrics

The primary advantage of LLM-based evaluators lies in their higher correlation with human judgment compared to traditional automatic metrics. Our analysis shows that the correlation between LLM-based evaluators and human assessors approaches the inter-rater reliability among human annotators, suggesting near-human-level performance in commit message evaluation. Furthermore,

LLM-based evaluation offers significant efficiency gains in both time and cost. While human evaluation required approximately six days for independent annotation of 800 commit messages in this study, LLM-based evaluation was completed within an hour. Cost analysis shows that using Chain-of-Thought and few-shot learning with unified evaluation criteria, GPT-4 evaluation averages US \$0.08 per example (calculated based on OpenAI API pricing⁵ and the average tokens used in this study), which is substantially more cost-effective than employing an experienced developer for six days. This improved efficiency enables rapid feedback integration during model development or training.

Despite these advantages, LLM-based evaluators exhibit non-deterministic behavior, producing varying scores for identical messages or semantically equivalent expressions. Although these variations remain within acceptable bounds and can be mitigated by averaging scores across multiple evaluations, further research is needed to improve evaluator consistency and address potential biases. Future work could explore ensemble approaches combining multiple LLMs with voting mechanisms or incorporate additional context beyond code changes to enhance evaluation reliability.

7.2 Threats to validity

The threats to **external validity** primarily concern the generalizability of our findings. The non-deterministic nature of LLMs may introduce instability and, thus, potentially misleading results. To mitigate this threat, we conducted stability analysis examining the effects of different temperature settings and tested LLMs with semantically equivalent messages expressed in varying ways. The results indicate that LLM-based metrics indeed exhibit some instability, though remaining within acceptable ranges. Due to this inherent variability, LLM-based metrics are unsuitable for precise comparative analyses and should instead be viewed as approximations of human judgment. For instance, if Model1 achieves an LLM-metric score of 3.78 on the *What* dimension while Model2 scores 3.71, this minor difference (relative to the standard deviation of 0.24 observed when using GPT-4 with temperature 0.25) should not be interpreted as definitive evidence of Model1’s superiority, but rather suggests that both models perform at comparable levels on this dimension. Another threat is that our stability measurement in RQ3 explored reproducibility, robustness, and fairness from limited perspectives. There are other factors that may affect LLM metrics’ reproducibility (e.g., software and hardware environment), and while our fairness evaluation did not observe significant bias when assessing semantically identical messages expressed differently, other comprehensive evaluations, such as assessing bias using semantically different examples that received consistent human ratings, should also be conducted (we only found 25 limited examples in this study). As a preliminary study, we only considered the significant factors, and a more comprehensive measurement of LLM metrics’ stability would require more extensive future work.

The threats to **internal validity** primarily relate to potential biases in human annotation. To address this concern, we recruited volunteers with an average of more than five years of software development experience and conducted a three-stage annotation

⁵<https://openai.com/api/pricing/>

process. Each volunteer independently evaluated the commit messages, and the Spearman correlation coefficients between each pair of volunteers' ratings demonstrated strong inter-annotator agreement. Another threat is our 30% threshold for high human ratings and low METEOR scores in our RQ2 thematic analysis. Higher thresholds would increase sample size but reduce accuracy by introducing more false positives. We chose 30% to balance meaningful pattern detection with a manageable yet discriminative sample of 102 examples.

7.3 Conclusion

This study presents a preliminary empirical investigation into utilizing Large Language Models (LLMs) for commit message quality evaluation. Our analysis demonstrates that incorporating Chain-of-Thought reasoning, few-shot learning, and unified evaluation criteria can enhance LLM-based evaluator performance. These evaluators substantially outperform all traditional automatic metrics in correlation with human judgment. Compared to human evaluation, LLM-based approaches offer superior efficiency in both time and cost. Integrating LLM-based evaluators in future commit message generation studies could facilitate better quality assessment and guide model development and training processes. While LLM-based evaluators show promising results, they present challenges regarding stability and potential biases. These limitations warrant careful consideration during implementation and necessitate further research for improvement.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Anonymous Author(s). 2025. Evaluating Generated Commit Messages with Large Language Models. <https://anonymous.4open.science/r/4k8asu>.
- [3] Satyanjeev Banerjee and Alon Lavie. 2005. METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments. In *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, Jade Goldstein, Alon Lavie, Chin-Yew Lin, and Clare Voss (Eds.). Association for Computational Linguistics, Ann Arbor, Michigan, 65–72. <https://aclanthology.org/W05-0909/>
- [4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [5] Rpl Buse and W. R. Weimer. 2010. Automatically documenting program changes. In *ASE 2010, 25th IEEE/ACM International Conference on Automated Software Engineering*. ACM, 33–42.
- [6] Raymond PL Buse and Westley R Weimer. 2010. Automatically documenting program changes. In *Proceedings of the 25th IEEE/ACM international conference on automated software engineering*, 33–42.
- [7] Cheng-Han Chiang and Hung yi Lee. 2023. Can Large Language Models Be an Alternative to Human Evaluations?. In *Annual Meeting of the Association for Computational Linguistics*. <https://api.semanticscholar.org/CorpusID:258461287>
- [8] Luis Fernando Cortés-Coy, Mario Linares-Vásquez, Jairo Aponte, and Denys Poshyvanyk. 2014. On automatically generating commit messages via summarization of source code changes. In *IEEE 14th International Working Conference on Source Code Analysis and Manipulation*, 275–284.
- [9] Daniela S Cruzes and Tore Dyba. 2011. Recommended steps for thematic synthesis in software engineering. In *2011 International Symposium on Empirical Software Engineering and Measurement*. IEEE Computer Society, 275–284. <https://doi.org/10.1109/ESEM.2011.36>
- [10] Jinhao Dong, Yiling Lou, Qihao Zhu, Zeyu Sun, Zhilin Li, Wenjie Zhang, and Dan Hao. 2022. FIRA: fine-grained graph-based code change representation for automated commit message generation. In *Proceedings of the 44th International Conference on Software Engineering*, 970–981.
- [11] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [12] Robert Dyer, Hoan Anh Nguyen, Hridesh Rajan, and Tien N Nguyen. 2013. Boa: A language and infrastructure for analyzing ultra-large-scale software repositories. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 422–431.
- [13] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- [14] Thong Hoang, Hong Jin Kang, David Lo, and Julia Lawall. 2020. CC2Vec: Distributed representations of code changes. In *ACM/IEEE 42nd International Conference on Software Engineering*, 518–529.
- [15] Xing Hu, Qiuyuan Chen, Haoye Wang, Xin Xia, David Lo, and Thomas Zimmermann. 2022. Correlating automated and human evaluation of code documentation generation quality. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 4 (2022), 1–28.
- [16] huggingface. 2025. Evaluate: A library for easily evaluating machine learning models and datasets. <https://github.com/huggingface/evaluate>.
- [17] Siyuan Jiang, Ameer Armaly, and Collin McMillan. 2017. Automatically generating commit messages from diffs using neural machine translation. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 135–146.
- [18] Xue Jiang, Yihong Dong, Lecheng Wang, Zheng Fang, Qiwei Shang, Ge Li, Zhi Jin, and Wenpin Jiao. 2024. Self-planning code generation with large language models. *ACM Transactions on Software Engineering and Methodology* 33, 7 (2024), 1–30.
- [19] Maurice G Kendall. 1938. A new measure of rank correlation. *Biometrika* 30, 1-2 (1938), 81–93.
- [20] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2024. Large language models are zero-shot reasoners. In *Proceedings of the 36th International Conference on Neural Information Processing Systems (New Orleans, LA, USA) (NIPS '22)*. Curran Associates Inc., Red Hook, NY, USA, Article 1613, 15 pages.
- [21] Udo Kuckartz, Stefan Rädiker, Thomas Ebert, and Julia Schehl. 2013. *Statistik: eine verständliche Einführung*. Springer-Verlag.
- [22] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 611–626.
- [23] Jiawei Li and Iftekhar Ahmed. 2023. Commit message matters: Investigating impact and evolution of commit message quality. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 806–817.
- [24] Jiawei Li, David Faragó, Christian Petrov, and Iftekhar Ahmed. 2024. Only diff Is Not Enough: Generating Commit Messages Leveraging Reasoning and Action of Large Language Model. *Proc. ACM Softw. Eng.* 1, FSE, Article 34 (July 2024), 22 pages. <https://doi.org/10.1145/3643760>
- [25] Chin-Yew Lin and Franz Josef Och. 2004. Automatic evaluation of machine translation quality using longest common subsequence and skip-bigram statistics. In *Proceedings of the 42nd annual meeting of the association for computational linguistics (ACL-04)*, 605–612.
- [26] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. DeepSeek-V3 Technical Report. *arXiv preprint arXiv:2412.19437* (2024).
- [27] Qin Liu, Ziheng Liu, Hongming Zhu, Hongfei Fan, Bowen Du, and Yu Qian. 2019. Generating commit messages from diffs using pointer-generator network. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories*. IEEE, 299–309.
- [28] Shangqing Liu, Cuiyun Gao, Sen Chen, Lun Yiu Nie, and Yang Liu. 2019. ATOM: Commit Message Generation Based on Abstract Syntax Tree and Hybrid Ranking. *CoRR* abs/1912.02972 (2019). [arXiv:1912.02972](http://arxiv.org/abs/1912.02972) <http://arxiv.org/abs/1912.02972>
- [29] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. 2023. G-Eval: NLG Evaluation using Gpt-4 with Better Human Alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 2511–2522. <https://doi.org/10.18653/v1/2023.emnlp-main.153>
- [30] Zhongxin Liu, Xin Xia, Ahmed E. Hassan, David Lo, Zhenchang Xing, and Xinyu Wang. 2018. Neural-machine-translation-based commit message generation: how far are we?. In *33rd ACM/IEEE International Conference on Automated Software Engineering, ASE 2018, Montpellier, France, September 3-7, 2018*. ACM, 373–384.
- [31] Pablo Loyola, Edison Marrese-Taylor, Jorge Balazs, Yutaka Matsuo, and Fumiko Satoh. 2018. Content aware source code change description generation. In *11th International Conference on Natural Language Generation*, 119–128.
- [32] Pablo Loyola, Edison Marrese-Taylor, and Yutaka Matsuo. 2017. A Neural Architecture for Generating Natural Language Descriptions from Source Code Changes. In *55th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Association for Computational Linguistics, Vancouver, Canada, 287–292.

- [33] Lun Yiu Nie, Cuiyun Gao, Zhicong Zhong, Wai Lam, Yang Liu, and Zenglin Xu. 2021. Coregen: Contextualized code representation learning for commit message generation. *Neurocomputing* 459 (2021), 97–107.
- [34] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*. 311–318.
- [35] N Reimers. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *arXiv preprint arXiv:1908.10084* (2019).
- [36] Maximilian Schall, Tamara Czinczoll, and Gerard De Melo. 2024. CommitBench: A benchmark for commit message generation. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 728–739.
- [37] Jinfeng Shen, Xiaobing Sun, Bin Li, Hui Yang, and Jiajun Hu. 2016. On automatic summarization of what and why information in source code changes. In *2016 IEEE 40th Annual Computer Software and Applications Conference*, Vol. 1. IEEE, 103–112.
- [38] Ensheng Shi, Yanlin Wang, Wei Tao, Lun Du, Hongyu Zhang, Shi Han, Dongmei Zhang, and Hongbin Sun. 2022. RACE: Retrieval-augmented Commit Message Generation. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (Eds.). Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 5520–5530. <https://doi.org/10.18653/v1/2022.emnlp-main.372>
- [39] Charles Spearman. 1961. The proof and measurement of association between two things. (1961).
- [40] Qwen Team. 2025. QwQ-32B: Embracing the Power of Reinforcement Learning. <https://qwenlm.github.io/blog/qwq-32b/>.
- [41] Yingchen Tian, Yuxia Zhang, Klaas-Jan Stol, Lin Jiang, and Hui Liu. 2022. What makes a good commit message?. In *Proceedings of the 44th International Conference on Software Engineering*. 2389–2401.
- [42] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [43] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [44] Ramakrishna Vedantam, C Lawrence Zitnick, and Devi Parikh. 2015. Cider: Consensus-based image description evaluation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 4566–4575.
- [45] Guoqing Wang, Zeyu Sun, Jinhao Dong, Yuxia Zhang, Mingxuan Zhu, Qingyuan Liang, and Dan Hao. [n. d.]. Is It Hard to Generate Holistic Commit Message? *ACM Transactions on Software Engineering and Methodology* ([n. d.]).
- [46] Guoqing Wang, Zeyu Sun, Jinhao Dong, Yuxia Zhang, Mingxuan Zhu, Qingyuan Liang, and Dan Hao. 2024. Is It Hard to Generate Holistic Commit Message? *ACM Trans. Softw. Eng. Methodol.* (Sept. 2024). <https://doi.org/10.1145/3695996> Just Accepted.
- [47] Haoye Wang, Xin Xia, David Lo, Qiang He, Xinyu Wang, and John Grundy. 2021. Context-Aware Retrieval-Based Deep Commit Message Generation. *ACM Transactions on Software Engineering and Methodology* 30, 4, Article 56 (2021), 30 pages.
- [48] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [49] Shengbin Xu, Yuan Yao, Feng Xu, Tianxiao Gu, Hanghang Tong, and Jian Lu. 2019. Commit Message Generation for Source Code Changes. In *Twenty-Eighth International Joint Conference on Artificial Intelligence*. 3975–3981.
- [50] Pengyu Xue, Linhao Wu, Zhongxing Yu, Zhi Jin, Zhen Yang, Xinyi Li, Zhenyu Yang, and Yue Tan. 2024. Automated Commit Message Generation With Large Language Models: An Empirical Study and Beyond. *IEEE Transactions on Software Engineering* 50, 12 (2024), 3208–3224. <https://doi.org/10.1109/TSE.2024.3478317>
- [51] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024. Qwen2. 5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024).
- [52] Linghao Zhang, Jingshu Zhao, Chong Wang, and Peng Liang. 2024. Using Large Language Models for Commit Message Generation: A Preliminary Study. *arXiv preprint arXiv:2401.05926* (2024).
- [53] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q Weinberger, and Yoav Artzi. 2019. Bertscore: Evaluating text generation with bert. *arXiv preprint arXiv:1904.09675* (2019).
- [54] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. 2020. BERTScore: Evaluating Text Generation with BERT. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=SkeHuCVFDr>
- [55] Yuxia Zhang, Zhiqing Qiu, Klaas-Jan Stol, Wenhui Zhu, Jiaxin Zhu, Yingchen Tian, and Hui Liu. 2024. Automatic commit message generation: A critical review and directions for future work. *IEEE Transactions on Software Engineering* (2024).
- [56] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223* (2023).